Assignment Mongodb Advanced

Task 1: Schema Validation and Data Creation

Step 1: Select a Fresh Database

```
test> use company_advanced
switched to db company_advanced
company_advanced>
```

Step 2: Create the Validated Collection

Command :

```
db.createCollection("employees", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "departmentId", "experience", "active"],
      properties: {
        name: {
          bsonType: "string"
        },
        departmentId: {
          bsonType: "int"
        },
        experience: {
          bsonType: "int"
        },
        active: {
          bsonType: "bool"
        }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
})
```

Output :

```
company_advanced> db.createCollection("employees", {
|   validator: {
|     $jsonSchema: {
|       bsonType: "object",
|       required: ["name", "departmentId", "experience", "active"],
|       properties: {
|         name: {
|           bsonType: "string"
|         },
|         departmentId: {
|           bsonType: "int"
|         },
|         experience: {
|           bsonType: "int"
|         },
|         active: {
|           bsonType: "bool"
|         }
|       }
|     }
|   },
|   validationLevel: "strict",
|   validationAction: "error"
| })
{ ok: 1 }
```

Step 3: Insert the Starting Dataset

```
company_advanced> db.employees.insertMany([
|   {
|     _id: 1,
|     name: "John",
|     departmentId: NumberInt(10),
|     skills: ["Java", "MongoDB"],
|     experience: NumberInt(4),
|     active: true,
|     certifications: [
|       {
|         name: "MongoDB",
|         status: "Expired",
|         expiryYear: NumberInt(2026)
|       }
|     ]
|   },
|   {
|     _id: 2,
|     name: "Alice",
|     departmentId: NumberInt(10),
|     skills: ["Python", "MongoDB"],
|     experience: NumberInt(6),
|     active: true,
|     certifications: [
|       {
|         name: "MongoDB",
|         status: "Active",
|         expiryYear: NumberInt(2028)
|       }
|     ]
```

Step 4: Test the Validation Rule

Trying to add invalid document.

Command :

```
db.employees.insertOne({
  _id: 99,
  name: 123,
  departmentId: "HR",
  experience: "one",
  active: "yes"
})
```

Output :

```
company_advanced> db.employees.insertOne({
|   _id: 99,
|   name: 123,
|   departmentId: "HR",
|   experience: "one",
|   active: "yes"
[| })
Uncaught:
MongoServerError: Document failed validation
Additional information: {
  failingDocumentId: 99,
  details: {
    operatorName: '$jsonSchema',
    schemaRulesNotSatisfied: [
      {
        operatorName: 'properties',
        propertiesNotSatisfied: [
          {
            propertyName: 'name',
            details: [
              {
                operatorName: 'bsonType',
                specifiedAs: { bsonType: 'string' },
                reason: 'type did not match',
                consideredValue: 123,
                consideredType: 'int'
              }
            ]
          },
          {
            propertyName: 'departmentId',
            details: [
              {
                operatorName: 'bsonType',
                specifiedAs: { bsonType: 'int' },
                reason: 'type did not match',
                consideredValue: 'HR',
                consideredType: 'string'
              }
            ]
          },
          {
            propertyName: 'experience',
            details: [
              {
                operatorName: 'bsonType',
                specifiedAs: { bsonType: 'int' },
                reason: 'type did not match',
                consideredValue: 'one',
                consideredType: 'string'
              }
            ]
          },
          {
            propertyName: 'active',
            details: [
              {
                operatorName: 'bsonType',
                specifiedAs: { bsonType: 'bool' },
                reason: 'type did not match',
                consideredValue: 'yes',
                consideredType: 'string'
              }
            ]
          }
```

Task 2: Advanced Array Queries and Updates

2.1 Find One Matching Array Object

Use $elemMatch to find employees whose certifications contain the same object with name: "MongoDB" and status: "Active".

Command :

```
db.employees.find({
  certifications: {
    $elemMatch: {
      name: "MongoDB",
      status: "Active"
    }
  }
})
```

Output :

```
company_advanced> db.employees.find({
|   certifications: {
|     $elemMatch: {
|       name: "MongoDB",
|       status: "Active"
|     }
|   }
| })
[
  {
    _id: 2,
    name: 'Alice',
    departmentId: 10,
    skills: [ 'Python', 'MongoDB' ],
    experience: 6,
    active: true,
    certifications: [ { name: 'MongoDB', status: 'Active', expiryYear: 2028 } ]
  }
]
```

2.2 Update One Matching Array Element

Use the positional $ operator to change John's MongoDB certification status from Expired to Active.
Command :

```
db.employees.updateOne(
 {
  _id: 1,
  "certifications.name": "MongoDB"
 },
 {
  $set: {
   "certifications.$.status": "Active"
  }
 }
)
```

Output :

```
company_advanced> db.employees.updateOne(
|   {
|     _id: 1,
|     "certifications.name": "MongoDB"
|   },
|   {
|     $set: {
|       "certifications.$.status": "Active"
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

2.3 Update Selected Array Elements
Use arrayFilters to change the status to Renewal Due for certification elements whose expiryYear is less than 2027.

Command :

```
db.employees.updateMany(
 {},
 {
  $set: {
   "certifications.$[cert].status": "Renewal Due"
  }
 },
 {
  arrayFilters: [
   {
    "cert.expiryYear": { $lt: 2027 }
   }
  ]
 }
)
```

Output :

```
company_advanced> db.employees.updateMany(
|   {},
|   {
|     $set: {
|       "certifications.$[cert].status": "Renewal Due"
|     }
|   },
|   {
|     arrayFilters: [
|       {
|         "cert.expiryYear": { $lt: 2027 }
|       }
|     ]
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Task 3: Bulk Write Operations

Use one bulkWrite() call to complete all three operations:
1. Increase John's experience by 1.
2. Set Alice's active value to false.
3. Insert Emma using the document below.

Command :

```
db.employees.bulkWrite([
  {
    updateOne: {
      filter: { _id: 1 },
      update: {
        $inc: {
          experience: 1
        }
      }
    }
  },
  {
    updateOne: {
      filter: { _id: 2 },
      update: {
        $set: {
          active: false
        }
      }
    }
  },
  {
    insertOne: {
      document: {
        _id: 4,
        name: "Emma",
        departmentId: NumberInt(20),
        skills: ["Excel"],
        experience: NumberInt(2),
        active: true,
        certifications: [
          {
            name: "Excel",
            status: "Active",
            expiryYear: NumberInt(2026)
          }
        ]
      }
```

```
  }
 }
])
```

Verify Result :

```
company_advanced> db.employees.findOne(
|   { _id: 1 },
|   {
|     _id: 0,
|     name: 1,
|     experience: 1
|   }
[| )
{ name: 'John', experience: 5 }
```

Task 4: Advanced Aggregation

Step 1: Create the Departments Collection

```
company_advanced> db.departments.insertMany([
|   {
|     _id: NumberInt(10),
|     name: "Engineering"
|   },
|   {
|     _id: NumberInt(20),
|     name: "HR"
|   }
[| ])
{ acknowledged: true, insertedIds: { '0': Int32(10), '1': Int32(20) } }
```

4.1 Join Employees with Departments

Command :

```
db.employees.aggregate([
 {
  $lookup: {
   from: "departments",
   localField: "departmentId",
   foreignField: "_id",
   as: "department"
  }
 },
 {
  unwind: "department"
 },
```

```
  {
    $project: {
      _id: 0,
      employeeName: "$name",
      departmentName: "$department.name"
    }
  },
  {
    $sort: {
      employeeName: 1
    }
  }
])
```

Output :

```
company_advanced> db.employees.aggregate([
|   {
|     $lookup: {
|       from: "departments",
|       localField: "departmentId",
|       foreignField: "_id",
|       as: "department"
|     }
|   },
|   {
|     $unwind: "$department"
|   },
|   {
|     $project: {
|       _id: 0,
|       employeeName: "$name",
|       departmentName: "$department.name"
|     }
|   },
|   {
|     $sort: {
|       employeeName: 1
|     }
|   }
[| ])
[
  { employeeName: 'Alice', departmentName: 'Engineering' },
  { employeeName: 'David', departmentName: 'HR' },
  { employeeName: 'Emma', departmentName: 'HR' },
  { employeeName: 'John', departmentName: 'Engineering' }
]
```

4.2 Count Employees for Each Skill
Command :

```
db.employees.aggregate([
  {
    unwind: "skills"
  },
  {
    $group: {
      _id: "$skills",
      count: {
        $sum: 1
      }
    }
  },
  {
    $sort: {
      count: -1,
      _id: 1
    }
  }
])
```

Output :

```
company_advanced> db.employees.aggregate([
|   {
|     $unwind: "$skills"
|   },
|   {
|     $group: {
|       _id: "$skills",
|       count: {
|         $sum: 1
|       }
|     }
|   },
|   {
|     $sort: {
|       count: -1,
|       _id: 1
|     }
|   }
[| ])
[
  { _id: 'MongoDB', count: 2 },
  { _id: 'Communication', count: 1 },
  { _id: 'Excel', count: 1 },
  { _id: 'Java', count: 1 },
  { _id: 'Python', count: 1 }
]
```

4.3 Return Active Employees and a Count
Use $facet to return both active employee names and the total active employee count in one pipeline.
Sort names ascending.

Command :

```
db.employees.aggregate([
  {
    $facet: {
      activeEmployees: [
        {
          $match: {
            active: true
          }
        },
        {
          $project: {
            _id: 0,
            name: 1
          }
        },
        {
          $sort: {
            name: 1
          }
        }
      ],
      activeEmployeeCount: [
        {
          $match: {
            active: true
          }
        },
        {
          $count: "total"
        }
      ]
    }
  }
])
```

Output :

```
[
  {
    activeEmployees: [ { name: 'Emma' }, { name: 'John' } ],
    activeEmployeeCount: [ { total: 2 } ]
  }
]
```

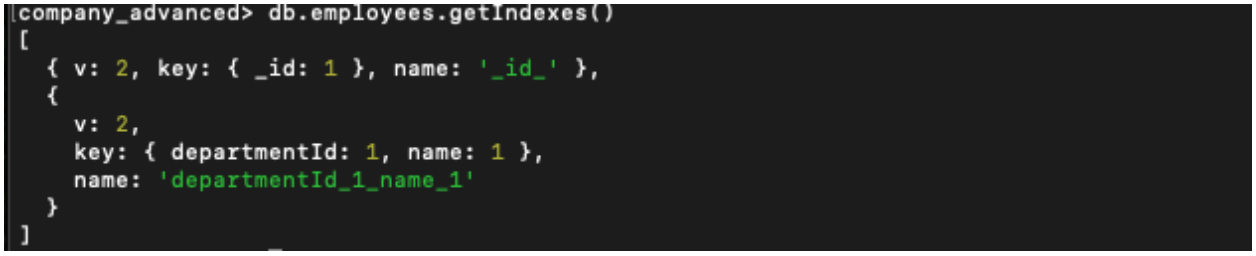
Task 5: Indexing and Query Performance

5.1 Create and Verify a Compound Index
Create an ascending compound index on departmentId followed by name. Verify it using getIndexes().

Command :

```
db.employees.createIndex(
  {
    departmentId: 1,
    name: 1
  },
  {
    name: "departmentId_1_name_1"
  }
)
```

Verify :



5.2 Read explain("executionStats")

Record the following values in answers.md:
● winning plan stage: COLLSCAN or IXSCAN
● nReturned
● totalDocsExamined
● totalKeysExamined

```
company_advanced> db.employees
| .find({ departmentId: 10 })
| .sort({ name: 1 })
| .explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'company_advanced.employees',
    parsedQuery: { departmentId: { '$eq': 10 } },
    indexFilterSet: false,
    queryHash: '1A8261F1',
    planCacheShapeHash: '1A8261F1',
    planCacheKey: '2AA77B48',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { departmentId: 1, name: 1 },
        indexName: 'departmentId_1_name_1',
        isMultiKey: false,
        multiKeyPaths: { departmentId: [], name: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { departmentId: [ '[10, 10]' ], name: [ '[MinKey, MaxKey]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 2,
    executionTimeMillis: 3,
    totalKeysExamined: 2,
    totalDocsExamined: 2,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 2,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 2,
        executionTimeMillisEstimate: 0,
```

5.3 Create a TTL Index

```
company_advanced> db.sessions.insertOne({
|   _id: 1,
|   userName: "John",
|   expiresAt: new Date(Date.now() + 600000)
[| })
{ acknowledged: true, insertedId: 1 }
[company_advanced>

[company_advanced> db.sessions.find()
[
  {
    _id: 1,
    userName: 'John',
    expiresAt: ISODate('2026-10-08T09:37:24.336Z')
  }
]
company_advanced> db.sessions.createIndex(
|   { expiresAt: 1 },
|   {
|     expireAfterSeconds: 0,
|     name: "expiresAt_1"
|   }
[| )
expiresAt_1
[company_advanced> db.sessions.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { expiresAt: 1 },
    name: 'expiresAt_1',
    expireAfterSeconds: 0
  }
]
```

Task 6: Backup and Restore

Use mongodump and mongorestore to complete the following steps:

1. Back up company_advanced to company_advanced.archive using gzip.

```
[prasad.pokale@Prasads-Mac-mini ~ % mongodump --db company_advanced --archive=company_advanced.archive --gzip
2026-10-08T15:04:49.666+0530    writing `company_advanced.employees` to "archive `company_advanced.archive`"
2026-10-08T15:04:49.666+0530    writing `company_advanced.sessions` to "archive `company_advanced.archive`"
2026-10-08T15:04:49.667+0530    writing `company_advanced.departments` to "archive `company_advanced.archive`"
2026-10-08T15:04:49.669+0530    done dumping `company_advanced.sessions` (1 document)
2026-10-08T15:04:49.677+0530    done dumping `company_advanced.employees` (4 documents)
2026-10-08T15:04:49.677+0530    done dumping `company_advanced.departments` (2 documents)
[prasad.pokale@Prasads-Mac-mini ~ % ls -lh company_advanced.archive
-rw-r--r--  1 prasad.pokale  staff   1.0K  8 Oct 15:04 company_advanced.archive
```

2. Restore it into a new database named company_advanced_restore.

```
prasad.pokale@Prasads-Mac-mini ~ % mongorestore \
  --archive=company_advanced.archive \
  --gzip \
  --nsFrom="company_advanced.*" \
  --nsTo="company_advanced_restore.*"
2026-10-08T15:05:28.427+0530    preparing collections to restore from
2026-10-08T15:05:28.435+0530    reading metadata for `company_advanced_restore.departments` from "archive `company_advanced.archive`
"
2026-10-08T15:05:28.435+0530    reading metadata for `company_advanced_restore.employees` from "archive `company_advanced.archive`"
2026-10-08T15:05:28.435+0530    reading metadata for `company_advanced_restore.sessions` from "archive `company_advanced.archive`"
2026-10-08T15:05:28.562+0530    restoring `company_advanced_restore.sessions` from "archive `company_advanced.archive`"
2026-10-08T15:05:28.574+0530    finished restoring `company_advanced_restore.sessions` (1 document, 0 failures)
2026-10-08T15:05:28.669+0530    restoring `company_advanced_restore.employees` from "archive `company_advanced.archive`"
2026-10-08T15:05:28.689+0530    finished restoring `company_advanced_restore.employees` (4 documents, 0 failures)
2026-10-08T15:05:28.784+0530    restoring `company_advanced_restore.departments` from "archive `company_advanced.archive`"
2026-10-08T15:05:28.796+0530    finished restoring `company_advanced_restore.departments` (2 documents, 0 failures)
2026-10-08T15:05:28.796+0530    no indexes to restore for collection `company_advanced_restore.departments`
2026-10-08T15:05:28.796+0530    restoring indexes for collection `company_advanced_restore.employees` from metadata
2026-10-08T15:05:28.796+0530    index: &idx.IndexDocument{Options:bson.M{"name":"departmentId_1_name_1", "v":2}, Key:bson.D{bson.E{K
ey:"departmentId", Value:1}, bson.E{Key:"name", Value:1}}, PartialFilterExpression:(*bson.D)(nil)}
2026-10-08T15:05:28.796+0530    restoring indexes for collection `company_advanced_restore.sessions` from metadata
2026-10-08T15:05:28.796+0530    index: &idx.IndexDocument{Options:bson.M{"expireAfterSeconds":0, "name":"expiresAt_1", "v":2}, Key:b
son.D{bson.E{Key:"expiresAt", Value:1}}, PartialFilterExpression:(*bson.D)(nil)}
2026-10-08T15:05:29.060+0530    7 document(s) restored successfully. 0 document(s) failed to restore.
```

3. Verify the restored collection counts.

```
test> use company_advanced_restore
switched to db company_advanced_restore
company_advanced_restore> show collections
departments
employees
sessions
company_advanced_restore> db.employees.countDocuments()
4
company_advanced_restore> db.departments.countDocuments()
2
company_advanced_restore> db.sessions.countDocuments()
1
```